The poster, explained to someone new

Read the plain text aloud. The grey lines in brackets are notes to you: where to point, or when to pause. Every number in here is on the poster or in the paper.

If you only have 20 seconds

This poster is about teaching a computer to learn from its mistakes the way people do. When it fails a puzzle, I want it to work out *why* it failed before it changes anything, because different failures need different fixes, and the wrong fix can break things that used to work. Measuring 60 real practice puzzles step by step, I found that the program's safety check works, accepting no wrong answers, and that its real problem is memorizing: it writes programs that fit the examples without capturing the rule. More search time does not fix that. So the program has not solved new puzzles yet. What it has given me is an exact diagnosis, and the next experiment builds the missing tool from that diagnosis.

Part 1. The story

Learning from failure is a very human thing

Think about how you learn. When you get something wrong, you do not just try the same thing again, harder. You ask: did I misread the question? Did I not know the method? Was there not enough information? Then you change *that* part. A child learning to stack blocks does this. A scientist whose experiment fails does this. A doctor does it too: diagnose first, then prescribe.

Most computer programs do not. When they fail, they either give up, or they do more of the same: search longer, try more guesses, use more computer power. My question is whether a program can do the human thing: notice *what kind* of failure it had, and change only the part that was actually missing.

The puzzles, and why they are a good test

[point to the small coloured grids, "demo 0" to "demo 3"]

The puzzles come from a test called ARC, created by the AI researcher François Chollet. Each puzzle is a small grid of coloured squares. You get a few examples, each one an input picture and the output picture it should turn into. You must work out the rule, and use it on a new picture.

Look at these four examples. Two yellow dots become a filled yellow square, and the dots mark its corners. So the rule is: draw the box whose corners are the dots, in the dots' colour. Most people see that in a few seconds.

Chollet's idea is that intelligence is not how many skills you already have. It is how quickly you can learn a *new* skill from very little. ARC is built to test exactly that: every puzzle has a new rule, and you usually get only two to five examples. You cannot pass it by memorizing. The version I use, ARC-AGI-2, is the harder second version. Chollet and Mike Knoop run the ARC Prize, which hosts this summit.

Where big neural networks struggle

Today's most famous AI systems are big neural networks: deep learning, the technology behind chatbots and image generators. They are excellent at things that look like what they saw in training. ARC is designed to be hard for them, for a few simple reasons:

- They learn from huge amounts of data. Here each new rule comes with only a handful of examples.
- They are good at recognizing familiar patterns, and weaker at a rule they have never seen.
- They have no built-in exact check, so they can be confidently wrong.

- When they fail, it is hard to see why, so it is hard to fix the right thing. The usual fix is more data, more training or more computing power.

To be fair: newer systems have improved a lot on the first ARC test, often by spending a great deal of computing on each puzzle, and many strong entries combine neural networks with search. Neural networks are not useless here. But on their own they do not tell you why they failed.

I took the opposite bet. My solver writes small, exact programs, and checks every program against every example. When it fails, it leaves a written record of what it tried. That record is something you can read and reason about. I also tried several neural pieces along the way: hints from a language model, learned ranking, a small recursive network and a search-guiding network. None of them helped enough in my setting, so I dropped them and wrote down why.

What others have done, and what is different here

Two well-known ideas come close to mine.

- **Counterexample-guided synthesis (CEGIS).** Guess a program, find an example where it breaks (a counterexample), use that to fix the guess, and repeat. It is "guess, check, fix".
- **DreamCoder** (Kevin Ellis and colleagues). It learns by writing programs and saving useful pieces as new building blocks, so its toolbox grows over time.

Both are important. My question is narrower: before fixing anything, decide *which layer* failed. Should the system search more, look at the picture differently, build a new tool, or admit that the examples are not enough? Then change only that layer, with checks that are fixed in advance and cannot be bent. That is a claim I can test and could lose. If my approach does not beat "guess, check, fix" and "grow a toolbox" under equal conditions, it is wrong. I have not run that comparison on real puzzles yet.

My journey: ten months of changing my mind

[optional: tell this part if the listener is interested; it is the human side of the work]

The work itself followed the same loop: try, fail, ask why, change the hypothesis.

- **December 2025.** A first prototype that used language-model hints. It was a prototype only.
- **April 2026.** I restarted with made-up puzzle worlds, where I know the answers. Checking every guess against the examples nearly eliminated false rules.
- **May to August 2026.** Real ARC puzzles. My solver went from 67 to 185 of the 1,000 practice puzzles, each one with a certificate.
- **The humbling part.** The same solver solved none of the 120 public test puzzles, and scored 0 on the online leaderboard. More hand-written tricks were clearly not enough. So I changed the goal: instead of adding skills by hand, could the system get what it was missing from its own failures?
- **September 2026.** The first try at that admitted nothing: 0 of 1,500 attempts. The reason was not bad luck. The new learner could only build what the old search could already reach.
- **Then a surprise.** The system's record of its own failures had been watching the wrong process, a simplified copy of the solver rather than the real one. Once repaired, the failures became informative.
- **Two opposite results.** Simply *describing* a failure did not help choose a fix (0.585 against 0.591, so no gain). Asking *how each possible fix changes the failure* did help (0.641 against 0.580). That told me failure is useful as something to experiment on, not just to describe.
- **October 2026.** On made-up puzzles the full loop worked, in 27 of 30. On real puzzles, it reached the solver for the first time, but did not survive the strict check.

- **Then a careful measurement.** I instrumented the solver on 60 real practice puzzles to see exactly where it fails. Its safety check is fine. The real problem is that it writes programs that fit the examples by memorizing them. The poster is about that finding, and what to build next.

Each step changed what I believed. That is the point: reasoning from failure is how people make progress, and I want a program that can do it too.

Part 2. The poster, panel by panel

1. The question

[“The question”, top left]

My question is: how can a program tell that its way of solving is not good enough, and get a better one, without cheating on the test that judges it? A failure can have different causes, and each needs a different fix:

- it did not look hard enough (*search*);
- it looked at the picture the wrong way (*representation*);
- it does not have the tool it needs (*capability*);
- the examples are not enough to pin down the rule (*evidence*);
- a correct answer was blocked by the safety checks (*certification*).

2. The CORA loop

[the diagram, left column]

CORA is the name of my system. The solver tries the puzzle. If it fails, I look at the record of what it did, decide what kind of failure it was, and choose a fix. The fix is tested with checks written down in advance, and only then can it be used. Solid boxes are built and measured. Dashed boxes are ideas not built yet. One honest point: *measuring* where the solver fails is now automatic (the “failure funnel”), but *choosing and applying* a fix automatically is built for only one kind of failure.

3. Four separate questions

[“Four claims, four kinds of evidence”]

I keep four questions apart:

- Did the new tool really cause the success?
- Did the record of the failure matter?
- Is the new tool truly new?
- Does it help later, on other puzzles?

The first two have early evidence on practice puzzles only. The third is not established. The fourth is not tested in this work.

4. What I measured

[the first bar chart, 27 / 30 against 12 / 30]

On made-up puzzles, the improved system completed 27 of 30, against 12 of 30 before. That is about made-up puzzles only.

[the second bar chart, which shrinks from 60 to 0]

On 60 real practice puzzles:

- the solver failed 54;
- in 31 it left useful clues about why;
- in 4, my system built a new tool from those clues and plugged it in;
- in 1, the solver accepted the tool and used it.

But that success failed the strictest check, called leave-one-out. I hide one example, rebuild everything from the rest, and see whether the hidden example still comes out right. It is like testing a student: if they can solve a worked example you hid from them, they understood it; if not, they may have just memorized. It passed 0 of 14 of these checks. So no new real puzzle was solved, and my official score stays at 185 out of 1,000.

5. Where it really fails

[the bar chart "Where CORA fails on real tasks", middle column]

I measured every step on 60 real practice puzzles. The hidden answers were checked only at the very end, by a separate program.

- **The safety check works.** Leave-one-out never accepted a wrong answer. If I switched it off, I would gain 2 right answers but add 26 wrong ones.
- **The big problem is memorizing.** In 27 puzzles (26 with its saved library loaded) the solver wrote programs that fit every example, and every one of them was wrong. They copy the exact squares that changed in each example, tied to that example's exact shape or colour, instead of learning the rule. It is like a student who memorizes the answers to the worked examples instead of the method. When I hide one example, the solver memorizes a different set.
- **More time does not help.** Three times the search time fixed none of them. The solver's own attempt to find a more general version found nothing.
- **Even the answer does not help.** When I gave it the hidden answer as one more example (a check only, never used as a prediction), it simply memorized one more case.
- **My earlier explanation was only a small part.** The "right view was cut" story is real, but it explains only 7 of the 76 failed checks.

In 22 more puzzles no program fit at all, mostly because the solver could not match the objects before and after.

6. Test a fix before you trust it

["Repairs tested before use", top right]

The obvious fixes were tested on made-up puzzles first, with the rules written down in advance. Forcing the solver to look at one particular view first helped none of 24 and hurt five, so I did not use it. Removing duplicate views fixed one of 41 and broke four, so I rejected it too. And every fix must play fair: when I hide an example to test the program, the program may not peek at it, not even indirectly.

7. Where the competition stands

["Competition status, 14 October"]

- **Official Kaggle score of my own solver, CORA: 0.00.**
- I also entered a strong neural method that someone else published (NVARC, a Qwen3-4B model that retrains itself briefly on each puzzle). I reproduced it, so it is *not* my invention, and its score is

not a CORA result. It processed every one of the 240 stand-in puzzles. Its official score is still being computed.

- On the 60 practice puzzles my whole solver got 12 right. But more than half of the answers it claimed were wrong (18 of 29), because its least careful part answers first. Simply reordering the answers it already makes gave 14 right, with none lost. That is a useful engineering fix, not a new way of reasoning, and I have not submitted it.

8. Building the missing tool

[the block "Constructing the missing operation", right column]

I built the missing tool. When the solver memorizes a growth pattern and its check rejects it, the system reads which objects it memorized. It then builds a general pattern rule from a fixed list of building blocks (37,750 possible rules), plugs it into the same solver, and tests it fairly: three times it hides one example, rebuilds the rule from the other two, and predicts the hidden one.

On made-up practice puzzles, one complete case worked. The solver failed on its own, the new rule was one it could not express before, all three hidden-example checks passed, and the final answer was right. Removing the rule broke it again, and giving the solver more search time instead fixed nothing. The rule it found is simple: a ring around the one odd-coloured cell. The puzzle was planted as a two-part rule, but on its shapes the second part never adds a cell, so this shows a missing one-step tool being built, not a complex one.

The honest part: a version that never looked at the failure record found the same rule. So this shows the system can build and check a missing tool. It does not yet show that learning from the failure is what made the difference. The bigger, sealed test is designed to check that next.

9. Could this work outside puzzles?

[the panel marked BIOLOGICAL APPLICATION: UNTESTED]

The decision-making part might work in other fields: diagnose the failure, choose a fix, check the fix. One example is predicting how cells respond to a drug or a gene change. The puzzle-specific parts would not carry over. This is only an idea: I have no biology data and no biology results.

10. The message

Fitting the examples is not understanding them. Before a reasoning system changes itself, it should find out what kind of failure it had, and every fix should be tested fairly, including on cases where it could do harm.

Part 3. What has been done, and what has not

Done and measured

- A puzzle solver that writes exact programs and certifies 185 of 1,000 ARC practice puzzles.
- On made-up puzzles: the full learn-from-failure loop worked in 27 of 30 cases, against 12 of 30 before.
- On 60 real practice puzzles: tools built from the solver's own failures reached the solver in 4, and one was accepted and used.
- A precise explanation of why that one failed the strict check: the right view was cut, not missing. At scale this explains only a small part of the failures (7 of 76 failed checks).
- A failure funnel on 60 real practice puzzles. The safety check accepted nothing wrong. The main failure is memorizing programs (27 puzzles), which neither more search time nor the solver's own

generalization step repairs.

- The whole solver on the same 60 puzzles: 12 right, with 18 of its 29 claims wrong. Reordering its answers: 14 right, none lost (engineering, not submitted).
- A reproduced neural baseline (NVARC's method, not CORA) scored 27.64 on Kaggle's official public leaderboard, the same as NVARC's own 2025 score. It shows our deployment is faithful, not that CORA works.
- A safety test showing that the obvious fix harms other puzzles, so it was not used.
- A test of whether a small AI model could pick the right view for the solver. It could not: the simple fixed order picked right 35 times out of 35, the AI 19 times, and only once when it was also shown the solver's failure record. A scrambled record did the same, so the record itself did not help.

Not done, or not yet

- No new real ARC puzzle solved, and no score improvement. The score is still 185 of 1,000 on practice puzzles, none of the 120 public test puzzles, and 0.00 on the official Kaggle leaderboard (October 2026).
- No proof that the system invents truly new ideas.
- Automatic failure diagnosis exists only as a first version. It reads recorded failures and agrees with the hand analysis in 13 of 18 cases, but it has not yet been tested on new failures.
- No fair comparison yet against "guess, check, fix" or "grow a toolbox" on real puzzles.
- The next experiment, constructing a general growth-pattern tool from failures, is designed but not run.
- The duplicate-removal fix was tested on made-up puzzles and rejected: it fixed one and broke four.
- A larger test on 120 new puzzles is frozen and has not been run.
- No biology results.

Part 4. If they ask

"Did it solve any new puzzles?" Not yet. No new real puzzles.

"Isn't this just guess, check, fix?" It includes that. What it adds is deciding what kind of failure happened before choosing a fix. Whether that extra step really helps is the next experiment.

"How is it different from DreamCoder?" DreamCoder grows a toolbox. I am asking when the toolbox is actually the problem, and when the problem is search, the view, or too few examples.

"Why not just use a big neural network?" They are powerful, but they need lots of data, they cannot check themselves exactly, and they do not tell you why they failed. I wanted failures I can read and fixes I can test.

"So was it a bug?" In this case, yes: a limit in how the solver chooses its views. Telling that apart from a missing ability is exactly what the diagnosis is for.

"What would Chollet say?" Probably: show me that it learns new skills efficiently, with every cost counted. That is the right test, and I have not passed it yet.

"What happens next?" Teach the system to notice when the examples are not enough to choose between competing fixes, instead of picking one, before any new real test.

Words you may need

ARC	a public set of picture puzzles that tests learning new rules from a few examples.
example pair	one input picture and the output it should become.
rule	what turns the input into the output.
solver, engine	the program that looks for the rule.
program synthesis	finding a rule by writing a small program.
neural network, deep learning	AI that learns patterns from large amounts of data.
view	one way of cutting a picture into objects.
leave-one-out	hide one example and check that the rest still give it correctly.
counterexample	a case where a guessed rule gives the wrong answer.
CEGIS	counterexample-guided synthesis: guess, check, fix, repeat.
DreamCoder	a system that grows its own toolbox of program pieces.
made-up (synthetic)	puzzles I generate myself, with known answers.
false acceptance	a wrong answer that passed the checks.

Part 5. How to reproduce what you see

- **Poster, abstract, explainer:** `sh docs/summit_poster/build.sh`. It redraws every figure in `docs/summit_poster/figures/` from hash-pinned recorded outputs (`scripts/summit_figures.py`; the hashes are in `figures/PROVENANCE.json`). Then run `python scripts/poster_to_pptx.py` for the PowerPoint.
- **The 90-second demo:** `sh docs/summit_demo_20261023/run_demo.sh`. This is an offline, hash-checked replay of the recorded run; its frames are written to its `frames/` folder.
- **The failure funnel:** `python -m cora_funnel.driver` runs it, and `python -m cora_funnel.evaluate` scores it. The record is `records/CORA_FAILURE_FUNNEL_20261014.md`.
- **The construction experiment:** `python -m cora_fhr.measure -corpus c4_dev_v2`, then `python -m cora_fhr.evaluate`. The witness audit is `python -m cora_fhr.witness`. The record is `records/CORA_FHR_DEV_RESULTS_20261014.md`.